2022 IEEE 19th International Conference on Software Architecture (ICSA) | 978-1-6654-1728-0/22/$31.00 ©2022 IEEE | DOI: 10.1109/ICSA53651.2022.00022

Architectural Refactoring for Functional Properties in Evolutionary Architecture

Nacha Chondamrongkul
School of Information Technology
Mae Fah Luang University
Chiang Rai, Thailand
nacha.cho@mfu.ac.th

Jing Sun
School of Computer Science
University of Auckland
Auckland, New Zealand
jing.sun@auckland.ac.nz

***Abstract*—The evolutionary architecture supports software systems to make small functional changes frequently and reliably. With fitness functions defined, we can ensure that the architectural goals are met as the system evolves. However, some functionality changes cause architectural changes, which impact a large part of the software system. Therefore, we aim at ensuring that changes in the architectural design do not impact the existing functionalities, while new functionalities are incorporated into the new design. This paper proposes an approach to automate architectural design refactoring that supports changes in functionalities. Our approach applies formal modeling and verification to refactor and verify the evolution process of software systems. The proposed algorithms help to automatically refactor the design by referencing the given architecture design. With our appproach, the evolution process can be planned and formally verified to guarantee that the system can evolve safely to support functionality changes. We evaluated our approach with four real-world systems. The results show the effectiveness of our refactoring approach to support new functional properties.**

***Index Terms*—Design Refactoring, Formal Method, Model Checking, Functional property, Evolutionary Architecture**

I. Introduction

Software systems require constant changes as they have to adapt to the changing operating environment. According to the previous study, [1], one of the major reasons for software changes is adding and/or updating system functionalities. Many of these functional changes require modification of the architectural design, which impacts a large part of the software system. Evolutionary architecture has been proposed as a principle to support guided constant changes [2]. Making small incremental changes over time through continuous delivery is the core idea of evolutionary architecture and a best practice of many modern systems [3]. With the fitness functions defined, we can ensure that the changes in architectural design can help to achieve the goals [4]. The fitness functions can be metrics that need to be evaluated manually or automated functions such as unit tests and integration tests that are executed by the deployment pipeline [5]. Although evolutionary architecture can help to serve constant changes, many challenges have been posed as follows. Firstly, the knowledge of current architecture and previous changes are required to accurately identify a design part to refactor [6]. Also, refactoring is a complicated task as we need to reason both structure and behaviour of the new design configuration. Secondly, the fitness functions that can be automated are usually in the form of test scripts, which require the implementation of changes to run. The change to implementation is costly since modification needs to be made to the source code. In addition, software refactoring is usually a repetitive task of resolving the issues and retesting [7]. It would be more cost-effective to prove that the design changes at the early stage are correct [8]. Thirdly, determining the evolution process consisting of a sequence of changes is difficult, as all architectural changes need to be thoroughly analysed together [9]. Moreover, as the system evolves, it is important to ensure that some existing functionalities and architectural characteristics are preserved. [10].

As this work focuses on design refactoring in evolutionary architecture to support functionality changes, we have carefully reviewed existing works in this area, which are discussed in Section VI. The findings can be concluded as follows. More works [11] [12] [13] [14] [15] have provided techniques to refactor the system implementation level rather than that of the design level. This finding aligns with the prior study [8], which found that refactoring at the model level is more challenging than code level as there are many views and aspects of design to be considered. Some works [16] [17] [18] [19] [20] provide techniques to improve non-functional requirement or system's quality attributes at the design level. However, none of these works address functional requirements in their design refactoring approaches. As the planning evolution process is a challenging task, some techniques [21] [22] [23] [24] have been provided. Although these techniques can well support planning the evolution of software, they focused more on organising changes at the implementation level.

This paper presents an architectural design refactoring approach to support changing functional requirements in evolutionary architecture. Formal techniques such as ontology reasoning and model checking are applied to automate refactoring and verification of the architectural design. The contribution of this work can be summarised as follows: 1) the refactoring algorithms are proposed to support different reconfiguration of architectural design. These algorithms are guided by the interactive behaviour derived from the reference architecture design. 2) the formal verification is used as a fitness function to ensure that the refactored architectural design can satisfy the functional requirements and its interactive behaviours are

978-1-6654-1728-0/22/$31.00 ©2022 IEEE
DOI 10.1109/ICSA53651.2022.00022

according to the desired architectural pattern. 3) a prototype tool has been developed to support performing modelling, refactoring and verification seamlessly. We have evaluated the effectiveness of our approach with the architectural design of four real-world software systems.

The rest of this paper is organised as follows. Section II presents the modelling and refactoring of software architecture design. Section III shows how to verify the refactored design and determine the evolution process. Section IV presents the supporting tools for our approach and how software engineers can use them in practice. Section V presents the evaluation outcome and the discussion. Section VI discusses the related works in comparison to our approach. This paper is concluded in Section VII with future research directions.

II. Automated Architectural Refactoring

Evolutionary architecture aims at reliably making small incremental changes in the software system over time. Therefore, our approach is developed to automate refactoring design and verify the functional changes, as shown in Figure 1.

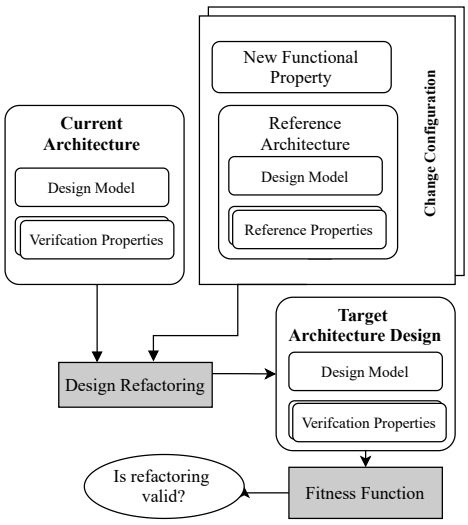

Fig. 1. Overall Process

Our approach applies formal techniques to software architecture design. As the fitness function is fundamental of evolutionary architecture, we apply formal verification [25] as a mean of fitness function. The overall process is shown in Figure 1. Firstly, the current design is formally modelled. The architecture design used in our approach consists of two major parts, namely, the design model and verification properties. The design model includes the structure and relationships among architectural design elements, as well as the architectural pattern that the design applies. The verification properties describe how the system should behave to serve the functionalities according to the desired architectural pattern. The model checker is used to verify the properties against the model to prove whether the system can satisfy defined properties. Secondly, a change configuration is specified, which includes the model of the reference design. The reference design is a template that provides the architectural structure to serve new functionalities, which are defined as reference properties. The model checker can also be used to simulate the reference design and verify its reference properties. The states trace is given as a result, which describes the sequence of executing components to satisfy the property. In change configuration, the new functional properties are also defined to describe changing or adding functionalities. Thirdly, the states trace given from the verification of reference properties is analysed to refactor the current design. As a result, the target architecture design is created. Lastly, the target design is verified to check the functional properties, which consists of existing functions to preserve and new functions. If the functional properties are proved valid against the target design, we can conclude that the target design can satisfy the desired functionalities. The properties of the architectural patterns are checked against the target design to ensure that it behaves according to the patterns. The following subsections explain the modelling of reference architecture, refactoring and verification of the refactored design in more details.

A. Modelling Software Architecture

Refactoring architectural design requires reasoning on both structural and behavioural aspects of software architecture design [26]. Therefore, we apply two modellings, namely structural modelling and behavioural modelling to describe the current design, reference design and target design in this work. These modellings are based on Component & Connector (C&C) view [27]. The Ontology Web Language (OWL) [28] is used to describe the structural model of the architectural design, whilst its expressiveness in describing the structural information. Architectural Description Language (ADL) is used to describe the interactive behaviour among architectural elements such as components and connectors.

For the structural model, the design elements in C&C view, such as component, connector, role and port, are defined as the ontology classes. Different types of ontology object properties are defined to describe the relationships among design elements as follows. A component may have one or more ports that serve as interfaces to other components. A connector may have one or more roles that specify the parties involved in the interaction. A port can be attached to one or more roles to form the interactive connectivity among components.The architectural design can be based on architectural patterns such as Client-Server (CS), Publish-Subscribe (PS), Data Repository (RP) and Event-Sourcing (ES). Mutiple connectors and components can be based on the same patterns that we predefined their behaviour and structure. We have formally defined these patterns [1] that can be reused in any design. The structural model of a software system is defined as ontology individuals based on the set of architectural patterns. The formal form of architectural pattern helps us automatically reason the design elements' types by their structure according to the patterns. The ontology reasoning [29] contributes to ensure that the structure of the refactored design conforms

[1]The formal architectural patterns can be found at https://bit.ly/2vGv6qi

with the patterns, before making behavioural modelling and verification in the next step.

The behavioural modelling used in this work is based on Wright# [30], as it is an ADL based on C&C view. Wright# provides a syntax that describes the interactive behaviour of component and connector. The behaviour of architectural patterns can also be formally specified and reused in the design. We can define the verification properties in Linear Temporal Logic (LTL) [31] to capture the expected interactive behaviour and use them to verify against the model. The verification can be performed using Process Analysis Toolkit (PAT) [32] as a model checker. We can define system functionalities as the properties and perform verification to ensure that functionalities can be satisfied by the design. The behaviour of architectural patterns that applies to the design can also be defined as the verification properties to ensure that the design behave according to the desired patterns.

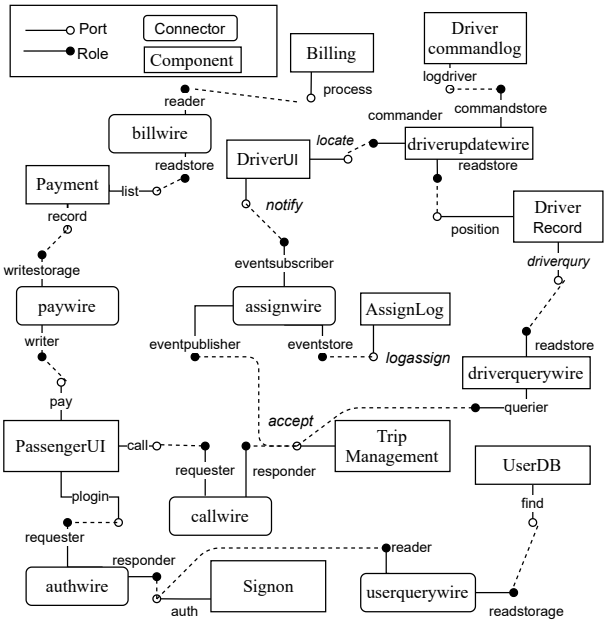

Fig. 2. Current Architecture Design of Ride Sharing

In this paper, we use the ride-sharing system (RSS) as an illustrative example. This system allows users to request a car hire from their location. The component and connector view of the current architectural design of RSS is shown in Figure 2. The rounded edges rectangles represent connectors, while the rectangle represents components. *PassengerUI* is a mobile application that allows users to request a ride. After the request is made, *TripManagement* component executes the request by assigning and notifying the available driver closest to the user's location. Then, the driver is notified on *DriverUI* with the details of the user that makes the request. After the users is picked up and arrives at the destination, they can make a payment transaction that will be recorded on the *Payment* component. The transactions are complete when *Billing* component processes the transactions, which is done periodically. The user can log on to the application through *SignOn* component, which looks up the user record stored on *UserDB*. This system has been implemented based on microservice architecture and applies various architectural patterns such as CS, RP and ES.

We have modelled RSS's design with the structural model and behavioural model[2]. For the structural model, the ontology individuals are defined for components, connectors, roles and ports that are in the design of RSS. Due to the page limit, we present some part of its structural model in Listing 1. *PassengerUI* is defined as an individual representing a component that is associated to ports: *call*, *pay* and *plogin*. *TripManagement* is associated to a port *accept*. *callwire* is defined as an individual representing a connector that has *requester* and *responder* as its roles. The *call* port is associated to *requester* role.

Listing 1. Structural Model

```
Individual(ex: PassengerUI
  value(ex: hasPort ex: call,pay,plogin))
Individual(ex: TripManagement
  value(ex: hasPort ex: accept))
Individual(ex: callwire
  value(ex: hasRole ex: requester,responder))
Individual(ex: call
  value(ex: hasAttachment ex: requester)))
```

Listing 2 shows some part of the behavioural model[3]. In this model, two types of connectors are defined namely *CSConnector* for Client-Server pattern and *ESConnector* for Event-Sourcing pattern. The behaviour of roles is defined as Communicating Sequential Process [33] with the events and channels to communicate among roles. *PassengerUI* and *DriverUI* are defined with their ports, which their behaviours consists of the sequence of events. The *rss* system is defined to describe how these design elements are connected. For example, *callwire* is defined as a connector assumming the behaviour of *CSConnector*, while *assignwire* is defined with *ESConnector*. The *call* port of *PassengerUI* is attached to *requester* role. The *notify* port of *DriverUI* is defined with *event subscriber* role. The system is executed by initiating the ports of components such as, *PassengerUI.call()* and *TripMgmt.accept()* in parallel.

Listing 2. Behavioural Model

```
connector CSConnector {
  role requester(j) = process -> req!j
       -> res?j -> Skip;
  role responder() = req?j -> invoke
       -> process -> res!j -> responder();}
connector ESConnector {
  role eventpublisher(j) = process
       -> pevt!j -> sevt?j
       -> bevt!j -> broadcast -> Skip;
  role eventsubscriber() = bevt?j
       -> process -> eventsubscriber(); ...
component PassengerUI {
  port call() = callride->call();
  port pay() = issuepay->pay();...
system RSS {
  declare callwire = CSConnector;
  declare assignwire = ESConnector;
```

[2]The structural model of RSS can be found at https://bit.ly/31s0M2j

[3]The behavioural model of RSS can be found at https://bit.ly/3jYPUiA

```
attach PassengerUI.call() =
      callwire.requester(50); ...
execute PassengerUI.call() ...
```

With this behavioural model, the verification properties can be specified to describe expected behaviour as liveness properties [34]. We can define the liveness properties in LTL according to the formula below:

$$\Box(SrcEvent \rightarrow \Diamond ResEvent)$$

where *SrcEvent* is a source event that is triggered when the system is executed to serve a function, and *ResEvent* is a response event that is triggered when the system is about to complete executing a function. For example, in RSS, we can specify the functional properties below in LTL representing the system functionalities as following.

$(e1)$ $\Box(PassengerUI.call.callride \rightarrow \Diamond DriverUI.notify.notified)$
$(e2)$ $\Box(TripMgmt.accept.ack \rightarrow \Diamond DriverDB.dvquery.quried)$
$(e3)$ $\Box(PassengerUI.plogin.login \rightarrow \Diamond SignOn.auth.authenticated)$
$(e4)$ $\Box(Billing.process.processed \rightarrow \Diamond Payment.list.listed)$

These functional properties can be explained as follows. (e1) A driver will be notified when a passenger calls. (e2) When a call is requested, the system looks up an available driver closest to the user. (e3) When the passenger logs in, his or her credential needs to be authenticated. (e4) The payment transactions need to be processed periodically. These properties have been validated against the model in ADL and proved to be valid. The negation of these properties gives the state trace that helps to observe how the system is executed to serve the function. Below is a partial states trace given from the verification of property e3.

init $\rightarrow$ *PassengerUI_plogin_logined*
$\rightarrow$ *PassengerUI_authwire_requester_process*
$\rightarrow$ *authwire_req*!38 $\rightarrow$ *authwire_req*?38
$\rightarrow$ *UserDB_find_queried*
$\rightarrow$ *UserDB_userquerywire_readstorage_process...*

This states trace represents a sequence of events, which can be classified into three types. First, the port event, which represents the event triggered by the port. It is in the format: [*Component*]_[*Port*]_[*Event*], such as *PassengerUI_plogin_logined* and *UserDB_find_queried*. Second, the role event, which is the event triggered by the role. It is in the format: [*Component*]_[*Connector*]_[*Role*]_[*Event*], such as *PassengerUI_authwire_requester_process* and *UserDB_userquerywire_readstorage_process*. Third, the channel event represents the event triggered by the channel that is used to communicate among roles in CSP. The channel event is used by the connector to communicate among its roles, such as *authwire_req*!38 and $\rightarrow$ *authwire_req*?38. The states trace produced by the verification of properties will be used to guide the design refactoring.

B. Refactoring by Referencing

With our approach, the reference design needs to be defined for refactoring. The reference design is a template that describes the structure of architectural elements to support new functions. It may be reused when the same function needs to be incorporated in different systems. The reference design is modelled in ADL, and the reference properties are defined as verification properties in LTL. Model checking is used to process the reference properties against the model of reference design. We apply PAT as a model checker to make the verification and generate the states traces of system execution. These state traces show the sequence of how components, ports, connectors and roles are executed to serve the functions, which is behavioural information, besides the structural information that we already have in the model. We used the state traces generated from the verification of reference properties to guide the design refactoring. Refactoring is performed according to the algorithms presented below. These algorithms process the states trace to reconfigure the design. The refactoring are performed in the way that the behaviour of refactored design imitates that of the reference design. As a result, the target design is generated and will be verified with respect to new functional properties. Our approach supports three types of refactoring as follows:

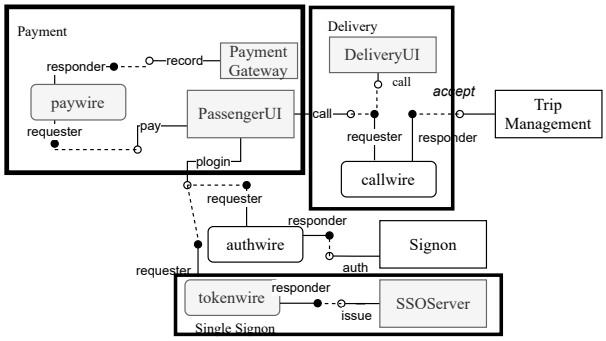

Fig. 3. Target Architecture of RSS

1) Configuration Replacement: handles the design configuration that needs to be replaced by a new configuration to support new functions. The refactoring procedure in Algorithm 1 takes a new functional property and the reference properties as inputs. The first loop iterates through the reference properties and fetches the state trace (*RefProp.statetrace*) to process by the inner loop. The events in the state trace are determined whether they are events triggered by the role of a connector (or role event). If it is a role event and the connector that triggers this event is not yet exist in the current design, a new connector is added. After that, we retrieve the previous event (*SrcEvent*) that is triggered by the port (or port event), using *backtrace()* function, which obtains the previous event in the state trace. If *SrcEvent* is a source event of the reference property *RefProp*, it is replaced with the source event of new functional property. If the component that triggers the *SrcEvent* event exists in the current design, it is replaced with a new component created according to the reference design. Otherwise, a new component is created for *SrcEvent*. The same steps are performed for the port event (*TgrEvent*), which is the next event in the state trace (fetched by *forthtrace()*). Finally, the new connector is attached to the replaced components.

Other connectors in the current design are reattached to the replaced components.

Algorithm 1 Replacing Design Configuration

```
1: FuncProp is a new functional property
2: RefProps is reference properties
3: for RefProp ∈ RefProps do
4:     RefProp.statetrace is state trace of this prop
5:     for Event ∈ RefProp.statetrace do
6:         if Event is an event triggerd by role then
7:             if Event.connector ∄ current design then
8:                 conn ← new Event.connector
9:             SrcEvent ← backtrace(event.component)
10:            if SrcEvent is a port event then
11:                if SrcEvent is a source event of RefProp then
12:                    SrcEvent ← FuncProp.sourceevent
13:                if SrcEvent.component ∃ current design then
14:                    comp ← new SrcEvent.component
15:                    replace SrcEvent.component with comp
16:                else
17:                    comp ← new SrcEvent.component
18:            TgrEvent ← forthtrace(event.component)
19:            if TgrEvent is a port event then
20:                if TgrEvent is a response event of RefProp then
21:                    TgrEvent ← FuncProp.responseevent
22:                if TgrEvent.component ∃ current design then
23:                    comp ← new TgrEvent.component
24:                    replace TgrEvent.component with comp
25:                else
26:                    comp ← new TgrEvent.component
27:            attach conn with comp
```

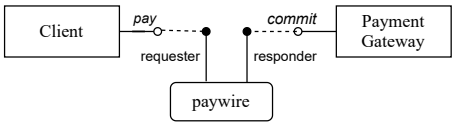

Fig. 4. Reference Architecture of Payment Gateway

For example, in RSS, the payment function needs to be changed to integrate with a payment gateway, instead of self-processing using *Payment* and *Billing* component as in the current design. The reference architecture of the payment gateway is defined as shown in Figure 4. It contains a reference property: $\square(Client.pay.preprocess \rightarrow \Diamond PaymentGateway.commit.checked)$. Below is a partial state trace generated by verifying this property against the model shown in Figure 4.

init → *Client_pay_preprocess*
→ *Client_paywire_requester_process*
→ *paywire_req*!91 → *paywire_req*?91
→ *PaymentGateway_paywire_responder_invoke*
→ *PaymentGateway_commit_checked*...

With this reference design, we define a new functional property as $\square(PassengerUI.pay.issuepay \rightarrow \Diamond PaymentGateway.commit.checked)$. Algorithm 1 iterates through events in the state trace of the reference design. The first role event found is *Client_paywire_requester_process*, where *paywire* is created as a new connector. After that, the procedure finds the port event *SrcEvent*, which is *Client_pay_preprocess* that is a source event of reference property. Then, *PassengerUI* is recreated with a new port, which is later attached to a new connector *paywire*. In the later iteration, the role event *PaymentGateway_paywire_responder_invoke* is processed, when *PaymentGateway* is created and attached to *paywire* connector. The result of this refactoring can be found as highlighted left top box in the Figure 3.

2) Configuration Extending: handles the design configuration that needs to be extended from the current design. This type of refactoring adds new architectural elements to the current design. Algorithm 2 explains how this type of refactoring is performed. The procedure takes a new functional property and the reference properties as inputs. The first loop iterates through the reference properties and fetches the state trace (*RefProp.statetrace*) to process by the inner loop. The inner loop iterates through the events in the state trace and finds the role event. Then, a new connector is created according to the role event when such a connector does not exist. After that, the algorithm retrieves the previous event (*SrcEvent*), which is a port event, using *backtrace()* function. If *SrcEvent* is a source event of the reference property *RefProp*, the procedure replaces it with that of new functional property. If the component that triggered *SrcEvent* exists in the current design, it is retrieved for attachment. Otherwise, the algorithm creates a new component for such a purpose. *TgrEvent* is processed with similar steps, i.e., the component that triggers it is created if not exists in the current design. At the last step, the new connector is attached to the new components.

Algorithm 2 Extending Design Configuration

```
1: FuncProp is a new functional property
2: RefProps is reference properties
3: for RefProp ∈ RefProps do
4:     RefProp.statetrace is state trace of this prop
5:     for Event ∈ RefProp.statetrace do
6:         if Event is an event triggerd by role then
7:             if Event.connector ∄ current design then
8:                 conn ← new Event.connector
9:             SrcEvent ← backtrace(event.component)
10:            if SrcEvent is a port event then
11:                if SrcEvent is a source event of RefProp then
12:                    SrcEvent ← FuncProp.sourceevent
13:                if SrcEvent.component ∃ current design then
14:                    comp ← SrcEvent.component
15:                else
16:                    comp ← new SrcEvent.component
17:            TgrEvent ← forthtrace(event.component)
18:            if TgrEvent is a port event then
19:                if TgrEvent is a response event of RefProp then
20:                    TgrEvent ← FuncProp.responseevent
21:                if TgrEvent.component ∃ current design then
22:                    comp ← TgrEvent.component
23:                else
24:                    comp ← new TgrEvent.component
25:            attach conn with comp
```

In RSS, the login function needs to be integrated with the Single Sign-On (SSO) server so that the users do not need to log in again when they have signed on to the third-party authentication service. The reference architecture of SSO

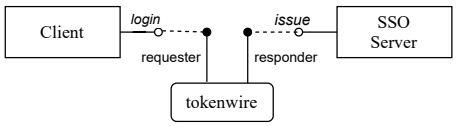

Fig. 5. Reference Architecture of SSO

can be seen in Figure 5. The reference property is defined as $\Box(\mathit{Client.login.submit} \rightarrow \Diamond \mathit{SSOServer.issue.tokenissued})$. This property gives the state trace as partly shown below.

init → *Client_login_submit*
→ *Client_tokenwire_requester_process*
→ *tokenwire_req*!10 → *tokenwire_req*?10
→ *SSOServer_tokenwire_responder_invoke*
→ *SSOServer_issue_tokenissued* → ...

SSO is added as an extension to the current design and the new functional property is defined as $\Box(\mathit{PassengerUI.ssologin.login} \rightarrow \Diamond \mathit{SSOServer.issue.tokenissued})$. The refactoring is performed to make this change. As a result, *SSOServer* is inserted as a new component. *tokenwire* is inserted and attached to *SSOServer* and *PassengerUI* according to the reference architecture and the new functional property. The refactored design can be seen in Figure 3 of the bottom highlighted box.

3) Configuration Mirrorring: is referred to when the current design is used as a reference architecture for refactoring. Mirrorring can be determined as configuration extending because it is used to extend the design configuration by replicating the existing function for different purposes. Thus, algorithm 2 can be used for mirrorring by specifying the current functional properties as the reference properties and the current design as the reference design.

For example, in RSS, we would like to add a new service that assists users to pick up ordered food at the restaurant and deliver it to the desired location. This new function is similar to the existing function. However, the pickup location and destination of a trip are reversed, so a new component *DeliveryUI* is required. At the level of architecture design, this component can replicate the structure and behaviour of *PassengerUI*. To support this new function, the current design is specified as the reference architecture and the functional property $\Box(\mathit{PassengerUI.call.callride} \rightarrow \Diamond \mathit{DriverUI.notify.notified})$ is specified as a reference property. The verification of this property produces the state trace (as partly shown below), which are used to perform the refactoring.

init → *PassengerUI_call_callride*
→ *PassengerUI_callwire_requester_process*
→ *callwire_req*!24 → *callwire_req*?24...
→ *DriverUI_notify_notified*...

The new functional property is defined as $\Box(\mathit{DeliveryUI.call.callride} \rightarrow \Diamond \mathit{DriverUI.notify.notified})$. According to Algorithm 2, the event *PassengerUI_callwire_requester_process* is the first role event that is procesed. The *callwire* is identified but not added, as it already exists in the current design. The *PassengerUI_call_callride* event is a port event that the algorithm determines as the source event of the reference property (at line 11). Therefore, *DeliveryUI.call.callride* as a source event of new functional property is processed instead. As a result, the *DeliveryUI* component is added to the design. Lastly, the *DeliveryUI* is attached to *callwire* connector. The result of this refactoring is shown as highlighted right top box in Figure 3.

With the reference architecture and new functional properties defined, refactoring can be performed automatically on the current design to create the target design. Our approach supports updating the existing functions and adding the new functions. The structure to support the existing function in the current design can also be used to create a new function. However, the target design may some time include unused components and connectors which are disintegrated from our refactoring algorithm. For example, the old *Payment* component that is replaced by *PaymentGateway* is still included in the design and disconnected from the rest of the system. We may use the technique proposed in [35] to detect the unused elements and remove them. As refactoring in the evolutionary architecture can frequently happen, automated refactoring is not sufficient as we need to also ensure that the refactoring is performed correctly and the other functions are still intact and not tampered by the refactoring.

III. Formal Fitness Function

In evolutionary architecture, fitness functions are defined to ensure that the architectural goals can be achieved [2]. In this work, fitness function is automated through formal verification of defined properties. The properties generally describe how the system should behave to support system functionality. The correctness of referactoring can be therefore checked by verifying the defined properties. After refactoring, we prepare a set of verification properties, which are selected from some properties in the existing architecture and new properties from the change configuration. The properties in this set may represent functionalities and architectural patterns that the updated design applies.

A. Functional Properties

After the target design is created through refactoring, the functional properties are verified for the following purposes: First, we verify existing functional properties to ensure that the functionalities that need to be preserved can still be satisfied by the target design. Second, we determine whether the target design can support new functionalities as it aims to.

($e1$) $\Box(\mathit{PassengerUI.call.callride} \rightarrow \Diamond \mathit{DriverUI.notify.notified})$
($e2$) $\Box(\mathit{TripMgmt.accept.ack} \rightarrow \Diamond \mathit{DriverDB.dvquery.quried})$
($e3$) $\Box(\mathit{PassengerUI.plogin.login} \rightarrow \Diamond \mathit{SignOn.auth.authenticated})$
($n1$) $\Box(\mathit{PassengerUI.pay.issued} \rightarrow \Diamond \mathit{Paym..Gateway.commit.checked})$
($n2$) $\Box(\mathit{PassengerUI.ssologin.login} \rightarrow \Diamond \mathit{SSOServer.issue.tokenissued})$
($n3$) $\Box(\mathit{DeliveryUI.call.callride} \rightarrow \Diamond \mathit{DriverUI.notify.notified})$

In RSS, the functionalities to be proved in the target design is as shown above. The property (e1),(e2) and (e3) are existing functional properties in the current design that needs to be preserved. While (e4) is replaced by (n1). (n2) and (n3) are

new functional properties. These new properties are previously explained in Section II-B.

We can use PAT-ADL to verify these functional properties against the model of target design as depicted in Figure 3. As the results shown below, these properties are proven to be valid, which guarantees that these functionalities can be satisfied with the design refactored by our approach.

$********VerificationResult********$
$TheAssertion(RSS() \models [](PassengerUI.call.callride...is\ VALID.$
$TheAssertion(RSS() \models [](TripMgmt.accept.ack...is\ VALID.$
$TheAssertion(RSS() \models [](PassengerUI.plogin.login...is\ VALID.$
$TheAssertion(RSS() \models [](PassengerUI.pay.issued...is\ VALID.$
$TheAssertion(RSS() \models [](PassengerUI.ssologin.login...is\ VALID.$
$TheAssertion(RSS() \models [](DeliveryUI.call.callride...is\ VALID.$

B. Architectural Properties

Architecture properties describe the interactive behaviours of different types of the component according to the architectural patterns. The architectural properties to be verified as fitness function can be derived from the current design or can be defined as a new property to check modified structure that supports new functionalities. The architectural properties can be defined according to the formula specific to architectural patterns, which are proposed in [36].

(1) $\Box([Client].[Prt].[Event] \rightarrow \Diamond[Server].[Conn].responder.process)$
(2) $\Box([DataReader].[Prt].[Event] \rightarrow \Diamond[Repository].[Conn].readstorage.process)$
(3) $\Box([EventSource].[Prt].[Event] \rightarrow \Diamond[EventStore].[Conn].eventlogger.persist)$

In the examples shown above, (1), (2), and (3) are the formula for defining properties of the client-server pattern, reading part of repository pattern and event-sourcing pattern, respectively. As these patterns are applied in the design of RSS, we can define the architectural properties to verify the target design as samples shown below. (a1) is the property derived from the current design to check the event sourcing pattern that is applied to *assignwire* connector. (a2),(a3) and (a4) are the properties defined for checking the behaviour of refactored configuration to support new functions.

$(a1)\ \Box(TripMgmt.accept.ack \rightarrow \Diamond AssignwireLog.assignwire.eventstore.persist)$
$(a2)\ \Box(PassengerUI.pay.issuepay \rightarrow \Diamond PaymentGateway.paywire.responder.process)$
$(a3)\ \Box(PassengerUI.ssologin.login \rightarrow \Diamond SSOServer.tokenwire.responder.process)$
$(a4)\ \Box(DeliveryUI.call.callride \rightarrow \Diamond TripMgmt.callwire.responder.process)$

After we have verified these architectural properties, the results are given as shown below. It can be observed that all architectural properties are proved valid. Therefore, we can conclude that the refactored design configurations can correctly behave according to the desired architectural patterns.

$********VerificationResult********$
$TheAssertion(RSS() \models [](TripMgmt.accept.ack...is\ VALID.$
$TheAssertion(RSS() \models [](PassengerUI.pay.issuepay...is\ VALID.$
$TheAssertion(RSS() \models [](PassengerUI.ssologin.login...is\ VALID.$
$TheAssertion(RSS() \models [](DeliveryUI.call.callride...is\ VALID.$

C. Evolution Planning

As the heart of evolutionary architecture is making small incremental changes over time, changes should not be applied all at once, but rather they should be implemented as an evolution process. The evolution process consists of a sequence of changes that need to be made to the system. The formal fitness function allows us to ensure that the evolution of software system can be conducted incrementally and safely.

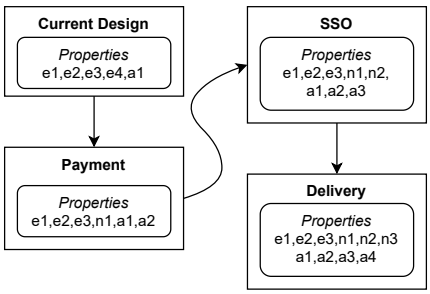

Fig. 6. Evolution Path of RSS

With the formal fitness function, we can check the evolution process by verifying necessary properties. A sample evolution process of RSS is shown in Figure 6. The properties *e1* to *e4* represents the existing functional properties in the current design, whilst *n1* to *n3* represents the new functional properties and a1 represents the architectural properties. In this evolution process, the current design has the functional property *e1* to *e4* and an architectural property *a1*. After the design is refactored to support new payment gateway, *e4* is replaced by *n1*, and *a2* is added as an architectural property. Then, the design from the previous stage is refactored to support the SSO function. The functional property *n2* and architectural property *a3* are added. At the last stage, the design is refactored to support the delivery function. The functional property *n3* and architectural property *a4* are added. The set of verification properties helps to ensure that the existing functionalities and architectural behaviours are preserved. Moreover, we can ensure that new design configurations are incorporated well in the overall design to support new functions.

IV. Implementation

The refactoring algorithms (as explained in Section 2.2) and verification steps (as explaied in Section 3) have been implemented as features in a modelling tool to support our approach. Our tool can automatically generated target design and its properties to check. This section explains the supporting tool and how to use it in practice.

A. Supporting Tool

We have implemented the refactoring algorithm as a part of ArchModeller[4], a software architecture modelling and verification framework. Figure 7 shows the overview of the framework that supports our approach. *ArchModeller* is used to graphically support the formal descriptions of the software architecture design, the verification properties and change configuration. The diagrams in *ArchModeller* are supported by the meta-model that we have defined. As *ArchModeller* is developed based on Eclipse Modelling Framework (EMF), the models created in the background are in the EMF format. Hence, we have developed *Model Formatter* as a module to convert the EMF model into ADL and OWL for structural and behavioural verification. *Design Refactorer* is developed as a module that takes the change configuration as input and refactors the current design to create the target design. *Structural Analyser* has an ontology reasoner [29] to perform structural verification, which compiles the design model in OWL according to the architectural pattern and identify the type of components and connectors included in the design. *Behavioural Analyser* has a model checker that verifies the functional and architectural properties against the design model in ADL. With the integration of these tools, modelling, refactoring and verification can be performed seamlessly.

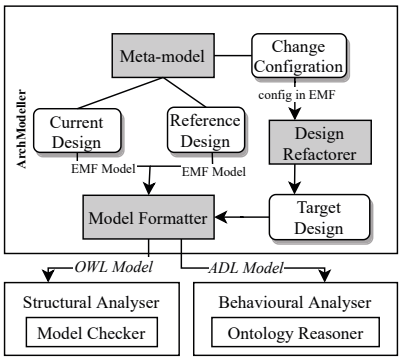

Fig. 7. ArchModeller with Design Refactorer

The drawing diagrams in ArchModeller are supported by the meta-model, which provides the structure of design elements in the diagrams. In this approach, we use three main diagrams as follows: *System diagram* illustrates the architectural design of a software system. *Change configuration diagram* illustrates the change configuration consisting of reference architecture and new functional properties. *Evolution diagram* illustrates the evolution process consisting of each stage's design model.

B. Use Case

With the enhanced ArchModeller framework to support design refactoring, software engineers can perform modelling, refactoring and verification seamlessly on the GUI of ArchModeller. The steps to use this software framework are as follows. Firstly, the software engineers draw the system diagram representing the current design. They define the functional and architectural properties of the current design. Then, the verification can be performed by the tool. The output of this step is the model of the current design with all properties verified. Secondly, the change configuration is defined using the change configuration diagram. The reference design model and its properties are created and verified in this step by software engineer. The refactoring type need to be defined in this step. The verification is then performed on the reference design to generate the state trace used for the refactoring. The new functional properties are also defined as a part of the change configuration. A change configuration can link to the other configuration to specify the sequence of refactoring in the evolution process. The output in this step is the sequence of change configurations that have the reference design and new functional properties defined. Thirdly, software engineers can execute the refactoring function with the tool, which gives the target design as a result. The target design can be verified with the tool. If there are any properties proved invalid, the refactoring is not performed as it is intended to; therefore, users need to check and revise reference design and properties as they could not be appropriately defined (more details discussed in Section V-B). The evolution process can be generated and can be viewed on the evolution diagram. The refactored design in each stage of the evolution process can also be viewed on the system diagram. Lastly, software engineers perform the verification on the refactored models. The results from the verification indicate whether the properties are proven to be valid as sample results previously shown in Section III. The results help to determine whether the evolution can be conducted safely to support new functional properties.

[4]Arch Modeller can be found at http://bit.ly/2m3LITT

V. Evaluation

This evaluation aims to assess the effectiveness of our approach to refactor the architectural design and how well it can be generalised to apply to different software systems. The definition of functional properties can be diverse; hence we have attempted to find the limitation of our refactoring approach. To our knowledge, there is no benchmark set to evaluate architectural design refactoring. Therefore, we chose four real-world software systems to conduct the experiment. Table I summarises information of the subject systems. *C&C* column indicates the number of components and connectors. *Func#* column indicates the number of functional requirements in the current design. The number of architectural properties is shown in column *Arch#*. *New#* shows the number of new functional requirements to be added or updated. The models used in this evaluation are not large; however, the overall size of model has little effect to the effectiveness of refactoring as our refactoring algorithms process only a particular portion of design that relates to the new functionalties. Also, the number of patterns applied to the design has no impact on the effectiveness of the refactoring algorithm since the refactoring relies on the state trace given from the verification of reference functional properties. In these models, the set of new functionalties of each system consists of all types of reconfiguration that supported by our approach to demonstrate that all algorithms are used.

TABLE I
SUBJECT SYSTEMS

System	C&C	Func#	Arch#	New#
Agri Digital (ADG) *Agriculture asset tracking system*	20	6	10	5
Life Net (LIN) *Medical emergency system*	19	5	9	7
Sock Shop (SSH) *E-Commerce System*	22	4	11	6
Ride Share (RSS) *Business process management*	18	8	4	6

A. Experiment Setup

We performed the evaluation according to the process described in Section IV-B, as we used ArchModeller to model and refactor the architecture design of subject systems[5]. After that, we perform refactoring to create a target design.

$$Recall = \frac{T_{pos}}{T_{pos} + F_{neg}} \quad Precision = \frac{T_{pos}}{T_{pos} + F_{pos}}$$

We evaluated the target design by making the verification of existing and new properties. The properties are defined to prove whether the target design can support desired functionalities. Therefore, we measure the completeness of the design refactoring by calculating the recall rate from the verification result, as the formulas shown above. The recall rate determines whether all parts are refactored as they are intended to. The true-positive result (T_{pos}) represents the property that is proved valid because the desired functional or architectural properties are correctly satisfied in the target design. The property that is proved to be invalid is indicated as false-negative (F_{neg}), as it cannot be satisfied in the target design as it should be. In addition, we evaluate the correctness of refactoring by investigating the cases when the properties are proven to be valid, but the structure of the target design does not meet the architect's expectations. Such cases are non-deterministic results and can be considered false-positive (F_{pos}). We have manually checked the target design to identify the F_{pos} results. The precision rate is calculated to measure the correctness of the refactoring algorithms.

Moreover, this evaluation also aimed at finding possible limitation of our approach in refactoring. We have investigated the false-negative cases. When such a result occurs, an isolated segment of the design configuration can be found in the target design. The isolated segment is a part of the design that does not connect to the rest of the system and can be removed; therefore, it is not considered a part of the system. This circumstance could be caused by unrealistic change configuration, which will be discussed in more detail in the following subsection.

B. Results & Discussion

The results of property verification have been calculated as recall rates, which are shown in Table II. According to the recall rates, it can be seen that the design of subject systems can be completely refactored since all functional properties and architectural properties are proven valid in the target design. However, we have found several refactoring mistakes in ADG and SSH, as it can be seen that their precision rates are less than 1.0.

TABLE II
SUMMARY OF VERIFICATION RESULTS

System	Recall	Precision
ADG	1.0	0.8
LIN	1.0	0.71
SSH	1.0	1.0
RSS	1.0	1.0

After we have investigated the false-positive results, we can conclude three scenarios as follows. 1) when the new functional property has the trigger event found in the reference design, but the response event does not exist in the current design. After refactoring, an isolated segment consisting of components and connectors according to the reference design can be found. However, they have no connectivity to the rest of the system. 2) when the new functional property has the trigger event that cannot be found in the current designs but the response event was found in the reference design. This problem occurs only when the reference design is not the current design (or it is not configuration mirrorring). 3) when the new functional property contains both response events and trigger events found in the reference design. Such property only refers to the events in the reference design and have no integration to any part of the current design. These three scenarios are invalid properties since they are not correctly defined to integrate the new configuration to the existing one. Another limitation is that our approach does not yet support refactoring that removes existing functions. Removing design configuration may impact the other design parts supporting other functionalities, so it must be carefully performed.

According to these observations, we can conclude that our refactoring approach works well under normal circumstances. First, when the reference design is the current design (configuration mirrorring), the new functional property must contain either a response event or trigger event that does not already exist in the current design. Second, when the current design is not used as the reference design, the new functional property must contain either trigger event or response event from the current design. To achieve higher precision of refactoring, we can add these restrictions as a set of rules to check before refactoring is performed.

C. Threats to Validity

There are limitations in our evaluation, which can be classified into internal threats and external threats to validity. *Internal threat* lies in the modelling of the architectural design of current architecture and reference architecture. In our evaluation, a group of software engineers helped to interpret the architectural design from available documents and create the model using ArchModeller. The misinterpretation of design into the model may create a gap between the model and the ground-truth architecture of the software system. The probability of having an inaccurate model has been minimised by

[5]The model of subject systems can be found at https://bit.ly/3q5oZpg

peer-reviewing. *External threat* is the variation of architectural design used in this evaluation. The software architecture of subject systems only represents distributed software systems based on the event-sourcing, client-server and repository patterns. The evaluation results are, therefore, only limited to these styles of system structure. More subject systems would be required for the evaluation to generalise our approach to suit other architecture styles.

VI. Related Work

Much research in software refactoring aims at improving the implementation of the software system in some aspects. The principle proposed by Fowler [11] has influenced many works that focused on eliminating software smells, which decay the maintainability of software systems. Search-based algorithms have been proposed to pinpoint whereabouts in the system to refactor. Rizzi et al. [12] applied a search-based algorithm to find the issues in the source code that impact the architectural design. Lin et al. [13] proposed an interactive system that guides the refactoring; however, the system relies on user feedback, which could be biased. Fadhel et al. [14] applied search-based technique to generate possible series of refactoring steps. However, their approach requires a current and target model to plan refactoring. Although these works support refactoring, they focus on refactoring at the implementation level, whereas ours focuses on refactoring at the design level.

Refactoring to enhance the quality attributes of the software system have been a focus of many works. Holmes and Zdun [16] proposed an automated approach to improve security and availability through architectural refactoring. However, many details about the cloud infrastructure need to be included in the model. Alshayeb et al. [17] applied a genetic algorithm to detect vulnerabilities and suggest refactoring in the UML sequence diagram. However, the detection algorithm relies heavily on a set of rules, and their approach only focuses on security at the implementation level. Cortellisa et al. [18] presented an approach that maps the UML model to Generalised Stochastic Petri Nets (GSPN) [37] to analyse availability in the design. Their approach can suggest refactoring in the model based on the fault tolerance patterns.

Some researches aim at enhancing and securing the evolution of software systems with formal techniques. Tanhaei et al. [21] proposed a refactoring framework that keeps software product lines (SPLs) [38] consistent with the feature model. The feature model is formally described to identify the refactoring pattern. However, their approach requires defining a customised algorithm specific to the pattern. Mokni et al. [22] proposed formal specification of architectural design to generate evolution plans. With a formal specification, the change requests can be analysed to generate the evolution plan. The architecture inconsistencies can be checked and avoided. However, this approach cannot guarantee that the functional requirements can be satisfied after the refactoring. Brito et al. [24] presented an algorithm to construct the refactoring graphs consisting of code changes over time. This approach can improve comprehension in changes to study software evolution; however, it only focuses on the changes at the code level and specific to a programming language.

In summary, none of the above existing works has been proposed to support refactoring at the architecture level that aims at achieving new functional properties. Most of the existing approaches focus on refactoring at the implementation level. Such tools and techniques to support refactoring at the implementation level are dedicated to a specific programming language or runtime platform. Moreover, most of the existing approach aims at improving non-functional requirements such as availability, maintainability and security, whereas ensuring the functional property has not been addressed. The functional requirements are our focus as our approach can automatically refactor the architectural design and ensure that both functional and architectural properties are satisfied. Additionally, to our knowledge, no existing works have been applied in the evolutionary architecture, in which the fitness function is required to determine changes in the software evolution. We have proposed formal verification as a fitness function to ensure that the evolution can be conducted safely. However, our model aims at describing the structural and behavioral aspects of architectural elements such as components and connectors since our approach focuses on refactoring at the architectural level. Therefore, some implementation details are not yet considered such as implementation constructs (e.g. class and interface), the network protocol used for communicating among components and supporting infrastructure. Other limitation is that our approach focuses at checking the functional properties. The functional property is defined to check a portion of the design that is responded to that functionality. Therefore, non-functional properties or system-wide quality attributes such as security, availability, and performance are not yet supported.

VII. Conclusion

This paper presents an automated solution that assists to refactor software architectural design to support new or changing functional requirements. Our approach applies formal modelling and verification to guarantee incremental changes over time, according to the concept of evolutionary architecture. With formal model of architectural design defined, the formal verification can be performed to produce the states trace used for guiding the refactoring in the design. Moreover, the formal verification is used to ensure that the functional and architectural requirements can be satisfied in the target design. We have implemented our approach as a tool so that the modelling, verification and refactoring can be performed seamlessly. After we evaluated our approach, the results from the experiment proved that our refactoring approach can be applied in real-world software systems.

For future work, we plan to apply the automated planning technique to evolution planning. The automated planning and formal verification can assist to automatically generate the plan for evolution process and enable us to identify when is the safe state that the system can evolve into.

REFERENCES

[1] M. Feathers, *Working Effectively with Legacy Code*. USA: Prentice Hall PTR, 2004.

[2] N. Ford, R. Parsons, and P. Kua, *Building Evolutionary Architectures: Support Constant Change*, 1st ed. O'Reilly Media, Inc., 2017.

[3] L. Chen, "Continuous delivery: Overcoming adoption challenges," *Journal of Systems and Software*, vol. 128, pp. 72–86, 2017. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0164121217300353

[4] A. Ramírez, J. R. Romero, and S. Ventura, "Interactive multi-objective evolutionary optimization of software architectures," *Information Sciences*, vol. 463-464, pp. 92–109, 2018. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0020025518304742

[5] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*, 1st ed. Addison-Wesley Professional, 2010.

[6] J. Ivers, I. Ozkaya, R. L. Nord, and C. Seifried, "Next generation automated software evolution refactoring at scale," in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2020. New York, NY, USA: Association for Computing Machinery, 2020, p. 1521–1524.

[7] A. M. Eilertsen, "Refactoring operations grounded in manual code changes," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Companion Proceedings*, ser. ICSE '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 182–185. [Online]. Available: https://doi.org/10.1145/3377812.3381395

[8] A. A. B. Baqais and M. Alshayeb, "Automatic software refactoring: a systematic literature review," *Software Quality Journal*, vol. 28, no. 2, pp. 459–502, Jun 2020. [Online]. Available: https://doi.org/10.1007/s11219-019-09477-y

[9] N. A. Ernst, A. Borgida, and J. Mylopoulos, "Requirements evolution drives software evolution," in *Proceedings of the 12th International Workshop on Principles of Software Evolution and the 7th Annual ERCIM Workshop on Software Evolution*, ser. IWPSE-EVOL '11. New York, NY, USA: Association for Computing Machinery, 2011, p. 16–20. [Online]. Available: https://doi.org/10.1145/2024445.2024450

[10] P. Di Francesco, P. Lago, and I. Malavolta, "Migrating towards microservice architectures: An industrial survey," in *2018 IEEE International Conference on Software Architecture (ICSA)*, 2018, pp. 29–2909.

[11] M. Fowler, *Refactoring: Improving the Design of Existing Code*, ser. A Martin Fowler signature book. Addison-Wesley, 2019.

[12] L. Rizzi, F. A. Fontana, and R. Roveda, "Support for architectural smell refactoring," in *Proceedings of the 2nd International Workshop on Refactoring*, ser. IWoR 2018. New York, NY, USA: Association for Computing Machinery, 2018, p. 7–10.

[13] Y. Lin, X. Peng, Y. Cai, D. Dig, D. Zheng, and W. Zhao, "Interactive and guided architectural refactoring with search-based recommendation," in *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, ser. FSE 2016. New York, NY, USA: Association for Computing Machinery, 2016, p. 535–546.

[14] A. ben Fadhel, M. Kessentini, P. Langer, and M. Wimmer, "Search-based detection of high-level model changes," in *2012 28th IEEE International Conference on Software Maintenance (ICSM)*, 2012, pp. 212–221.

[15] K. Paltoglou, V. E. Zafeiris, N. Diamantidis, and E. Giakoumakis, "Automated refactoring of legacy javascript code to es6 modules," *Journal of Systems and Software*, vol. 181, p. 111049, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0164121221001461

[16] T. Holmes and U. Zdun, "Refactoring architecture models for compliance with custom requirements," in *Proceedings of the 21th ACM/IEEE MODELS '18*. New York, NY, USA: Association for Computing Machinery, 2018, p. 267–277.

[17] M. Alshayeb, H. Mumtaz, S. Mahmood, and M. Niazi, "Improving the security of uml sequence diagram using genetic algorithm," *IEEE Access*, vol. 8, pp. 62 738–62 761, 2020.

[18] V. Cortellessa, R. Eramo, and M. Tucci, "From software architecture to analysis models and back: Model-driven refactoring aimed at availability improvement," *Information and Software Technology*, vol. 127, p. 106362, 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0950584920301300

[19] A. Martens, H. Koziolek, S. Becker, and R. Reussner, "Automatically improve software architecture models for performance, reliability, and cost using evolutionary algorithms," in *Proceedings of the First Joint WOSP/SIPEW International Conference on Performance Engineering*, ser. WOSP/SIPEW '10. New York, NY, USA: Association for Computing Machinery, 2010, p. 105–116.

[20] O. Deryugina, E. Nikulchev, I. Ryadchikov, S. Sechenev, and E. Shmalko, "Analysis of the anywalker software architecture using the uml refactoring tool," *Procedia Computer Science*, vol. 150, pp. 743–750, 2019, proceedings of the 13th International Symposium "Intelligent Systems 2018" (INTELS'18), 22-24 October, 2018, St. Petersburg, Russia. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1877050919303485

[21] M. Tanhaei, J. Habibi, and S.-H. Mirian-Hosseinabadi, "A feature model based framework for refactoring software product line architecture," *Journal of Computer Science and Technology*, vol. 31, 09 2016.

[22] A. Mokni, C. Urtado, S. Vauttier, M. Huchard, and H. Y. Zhang, "A formal approach for managing component-based architecture evolution," *Science of Computer Programming*, vol. 127, pp. 24 – 49, 2016, special issue of the 11th International Symposium on Formal Aspects of Component Software.

[23] K. Djibo., M. Oussalah., and J. Konate., "Evolution style mining in software architecture," in *Proceedings of the 15th International Conference on Evaluation of Novel Approaches to Software Engineering - ENASE,*, INSTICC. SciTePress, 2020, pp. 313–322.

[24] A. Brito, A. Hora, and M. T. Valente, "Refactoring graphs: Assessing refactoring over time," in *2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2020, pp. 367–377.

[25] C. Baier and J.-P. Katoen, *Principles of Model Checking (Representation and Mind Series)*. The MIT Press, 2008.

[26] O. Zimmermann, "Architectural refactoring for the cloud: a decision-centric view on cloud migration," *Computing*, 10 2017.

[27] D. Garlan, F. Bachmann, J. Ivers, J. Stafford, L. Bass, P. Clements, and P. Merson, *Documenting Software Architectures: Views and Beyond*, 2nd ed. Addison-Wesley Professional, 2010.

[28] G. Antoniou, , G. Antoniou, G. Antoniou, F. V. Harmelen, and F. V. Harmelen, "Web ontology language: Owl," in *Handbook on Ontologies in Information Systems*. Springer, 2003, pp. 67–92.

[29] B. Glimm, I. Horrocks, B. Motik, G. Stoilos, and Z. Wang, "Hermit: An owl 2 reasoner," *J. Autom. Reason.*, vol. 53, no. 3, pp. 245–269, Oct. 2014.

[30] N. Chondamrongkul, J. Sun, and I. Warren, "Pat approach to architecture behavioural verification," in *31th International Conference on Software Engineering and Knowledge Engineering*, July 2019, pp. 187–192.

[31] K. Y. Rozier, "Linear temporal logic symbolic model checking," *Computer Science Review*, vol. 5, no. 2, pp. 163–203, 2011. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1574013710000407

[32] J. Sun, Y. Liu, J. S. Dong, and C. Chen, "Integrating specification and programs for system modeling and verification," in *Proceedings of the third IEEE International Symposium on Theoretical Aspects of Software Engineering (TASE'09)*. IEEE Computer Society, 2009, pp. 127–135.

[33] C. A. R. Hoare, "Communicating sequential processes," *Commun. ACM*, vol. 21, no. 8, pp. 666–677, Aug. 1978.

[34] A. P. Sistla, "Safety, liveness and fairness in temporal logic," *Formal Aspects of Computing*, vol. 6, no. 5, pp. 495–511, Sep 1994. [Online]. Available: https://doi.org/10.1007/BF01211865

[35] N. Chondamrongkul, J. Sun, I. Warren, and S. U.-J. Lee, "Integrated formal tools for software architecture smell detection," *International Journal of Software Engineering and Knowledge Engineering*, vol. 30, no. 06, pp. 723–763, 2020.

[36] N. Chondamrongkul, J. Sun, and I. Warren, "Software architectural migration: An automated planning approach," *ACM Trans. Softw. Eng. Methodol.*, vol. 30, no. 4, jul 2021. [Online]. Available: https://doi.org/10.1145/3461011

[37] M. Marsan, G. Balbo, G. Chiola, G. Conte, S. Donatelli, and G. Franceschinis, "An introduction to generalized stochastic petri nets," *Microelectronics Reliability*, vol. 31, no. 4, pp. 699–725, 1991. [Online]. Available: https://www.sciencedirect.com/science/article/pii/0026271491900105

[38] L. Marchezan, E. Rodrigues, W. K. G. Assunção, M. Bernardino, F. P. Basso, and J. Carbonell, "Software product line scoping: A systematic literature review," *Journal of Systems and Software*, vol. 186, p. 111189, 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0164121221002673